Anexo 4A

Tipos en React (JSX → TSX)

Curso: TypeScript

150 min

Objetivos de Aprendizaje

Parte 1 - Tipos Avanzados de TypeScript:

- Aplicar Utility Types (Partial, Omit, Pick, Record)
- Implementar Generic Constraints con extends
- Utilizar Template Literal Types
- Aplicar Mapped Types
- Comprender Type Inference
- Usar el operador satisfies

Parte 2 - React + TypeScript:

- Migrar componentes de JSX a TSX
- Tipar props y children (ReactNode)
- Tipar state con useState<T>
- Tipar eventos de formulario
- Usar hooks con tipos (useRef, useCallback, useMemo)
- Crear componentes genéricos con constraints
- Aplicar React 18 con useId

1. Contexto: Tipos avanzados

15 minutos

El problema: Tipos inflexibles

```
interface Task {
  id: string;
  title: string;
  status: 'pending' | 'inProgress' | 'completed';
}

// ¿Cómo creo un tipo para tarea PARCIALMENTE editable?
// ¿Cómo creo un tipo sin el campo id?
// ¿Cómo creo un tipo solo con los campos de búsqueda?
```

Solución: Utility Types

```
type TaskPartial = Partial<Task>;     // Todos los campos opcionales
type TaskSinId = Omit<Task, 'id'>;    // Sin el campo id
type TaskPreview = Pick<Task, 'id' | 'title'>; // Solo algunos campos
type TaskMap = Record<string, Task>;  // Objeto con claves string
```

2. Utility Types

25 minutos

Partial, Required, Omit, Pick

Utility	Uso
Partial<T>	Todos los campos opcionales
Required<T>	Todos los campos obligatorios
Omit<T, K>	Excluir campos
Pick<T, K>	Seleccionar campos
Record<K,V>	Índice/Map

Lab: Utility Types

1. Crear tipo para formulario de edición (Partial)
2. Crear tipo para respuesta sin id (Omit)
3. Crear índice de tareas por prioridad (Record)

3. Generic Constraints

20 minutos

extends para limitar genéricos

```
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const task = { title: 'Learn TS', priority: 'high' };
getProp(task, 'title');    // ✅ string
getProp(task, 'priority'); // ✅ string
getProp(task, 'status');   // ❌ Error: no existe
```

Lab: Generic Constraints

1. Crear componente genérico con constraint
2. Usar extends para asegurar que tiene id

4. Template Literal Types

15 minutos

Crear tipos de strings específicas

```
type APIRoute = `/api/${string}`;
const taskRoute: APIRoute = '/api/tasks'; // ✅
const invalid: APIRoute = '/other';       // ❌

type TaskEvent = 'onClick' | 'onSubmit';
type Handler = { [K in TaskEvent]?: () => void };
```

Lab: Template Literal Types

1. Tipar acciones de tareas
2. Crear tipos para eventos

5. Mapped Types

15 minutos

Transformar tipos automáticamente

```
type Readonly<T> = { readonly [K in keyof T]: T[K] };
type Optional<T> = { [K in keyof T]?: T[K] };
type Nullable<T> = { [K in keyof T]: T[K] | null };
```

Lab: Mapped Types

1. Hacer todos los campos opcionales
2. Agregar prefijos

6. Type Inference

10 minutos

Inferencia en funciones

```
// TS infiere el tipo de retorno
function getTask() {
  return { id: '1', title: 'Learn TS' };
}
// Infiere: { id: string; title: string }

// as const - preservar literales
const colors = ['red', 'blue'] as const;
// readonly ['red', 'blue']
```

7. Satisfies Operator

10 minutos

satisfies vs as

```
// as - pierde literal
const c1 = 'debug' as string; // c1: string

// satisfies - mantiene literal + valida
const c2 = 'debug' satisfies 'debug' | 'info' | 'error';
// c2: 'debug' (literal)
```

Ejercicio Integrador – Parte 1

Aplicar conceptos al Caso 2r (Biblioteca)

Requisitos

- Utility Types: Partial, Omit, Pick
- Generic Constraints con extends
- Template Literal Types
- Mapped Types
- Satisfies

8. Migración JSX → TSX

15 minutos

De JSX a TSX

```
// TSX - con tipos explícitos
interface BookCardProps {
  titulo: string;
  autor: string;
  año?: number;
}

function BookCard({ titulo, autor, año }: BookCardProps) {
  return (
    <div>
      <h3>{titulo}</h3>
      <p>{autor} {año && `- ${año}`}</p>
    </div>
  );
}
```

9. Tipado de Props y Children

15 minutos

Props con Children

```
import { ReactNode } from 'react';

interface CardProps {
  title: string;
  children: ReactNode;
  footer?: ReactNode;
}
```

ReactNode vs otros tipos

Tipo	Qué acepta
ReactNode	Todo
ReactElement	Solo elementos JSX
string	Solo texto

10. Tipado de State

15 minutos

useState con genéricos

```
// Inferencia automática
const [task, setTask] = useState({ title: 'Learn' });

// Explícito
const [tasks, setTasks] = useState<Task[]>([]);
const [task, setTask] = useState<Task | null>(null);
```

11. Tipado de Eventos

15 minutos

Eventos de formulario

```
// Input change
onChange={(e) => setValue(e.target.value)}
// e: ChangeEvent<HTMLInputElement>

// Form submit
onSubmit={(e) => e.preventDefault()}
// e: FormEvent<HTMLFormElement>
```

12. Hooks Básicos Tipados

15 minutos

useRef, useCallback, useMemo

```
// useRef
const inputRef = useRef<HTMLInputElement>(null);

// useCallback
const handleClick = useCallback(() => { ... }, []);

// useMemo
const filtered = useMemo(() => tasks.filter(...), [tasks]);
```

13. Componentes Genéricos

15 minutos

Componentes con tipo genérico

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => ReactNode;
}

function List<T>({ items, renderItem }: ListProps<T>) {
  return <ul>{items.map(renderItem)}</ul>;
}
```

14. IDs Únicos en Componentes

10 minutos

useId para accesibilidad y SSR

```
const id = useId();
return (
  <div>
    <label htmlFor={id}>Nombre</label>
    <input id={id} />
  </div>
);
```

Ejercicio Integrador – Parte 2

Componentes React Tipados - Caso 2r

Requisitos

- BookCard: props tipadas con children
- BookList: componente genérico
- LoanForm: eventos tipados
- useState, useRef, useCallback
- useId para accesibilidad

15. Resumen

5 minutos

Puntos Clave

1. **Utility Types:** Partial, Omit, Pick, Record
2. **Generic Constraints:** extends para limitar tipos
3. **Template Literal Types:** `${}` para strings
4. **Mapped Types:** [K in keyof T]
5. **Type Inference:** Cómo TS infiere tipos
6. **Satisfies:** Validar sin perder literales
7. **JSX → TSX:** Migración de componentes
8. **Props y Children:** ReactNode
9. **State:** useState<T>

10**Eventos:** ChangeEvent, FormEvent

11**Hooks:** useRef, useCallback, useMemo

12**Componentes Genéricos:** <T> con constraints

13**useId:** IDs únicos para accesibilidad

16. Referencias

Recursos Adicionales

- TypeScript Utility Types
- TypeScript Generics
- Template Literal Types
- satisfies Operator
- React TypeScript Cheatsheet
- React 18 with TypeScript
- useId Hook

Gracias!

Anexo 4A Completado

Siguiente: Anexo 5A - SSR